C++ Programming Problem Set

Module: Reference Variables (Cumulative & Progressive Practice)

CS-SIMPLIFIED

Author: Gaurav Kandpal

Target Concept: Reference Variables (&)

Integrates: I/O, Operators, Loops, Functions, Arrays, Strings

Instructions & Pedagogical Guidelines

This problem set tests your mastery of C++ reference variables. You are required to solve problems that build cumulatively—combining reference syntax with earlier concepts like conditional statements, iteration, modular functions, array manipulation, and C++ strings. Pay careful attention to non-rebindability, pass-by-reference mechanics, and return-by-reference safety.

Level 1: Fundamentals & Direct Alias Manipulation

Problem 1: Memory Tracker & Value Synchronizer

Basic I/O | Operators | Reference Declaration

Write a C++ program that accepts two integers from the user, `valA` and `valB`. Create a reference variable `refA` bound to `valA`. Perform a sequence of operations exclusively through `refA` (e.g., multiply `refA` by 3, subtract `valB`). Finally, display the memory addresses of both `valA` and `refA` alongside their final values to prove they share physical memory.

Reference Syntax | Address-of Operator | Arithmetic Operators

Sample Input	Sample Output
`valA = 5, valB = 4`	`refA mutated: 11 | valA: 11` `&valA == &refA: TRUE (0x7ffd9000)`

Problem 2: Non-Rebinding Value Swap Simulator

Conditionals | Reference Behavior | Value Copying

Declare two integer variables `x = 10` and `y = 25`, and a reference `ref = x`. Prompt the user to enter a third number `z`. If `z` is positive, execute the assignment `ref = y`. Output the values of `x`, `y`, and `ref`, and explain why `ref` did *not* rebind to `y`, but instead altered `x`.

Non-rebindability | If-Else Conditionals | Value Copy vs Rebinding

Sample Input	Sample Output
`z = 12`	`x: 25, y: 25, ref: 25` `Note: ref remains bound to x; value of y was copied into x.`

Problem 3: In-Place Multi-Value Swapper (Pass-By-Reference)

Functions | Pass-by-Reference | Standard Swap

Write a function `void cyclicSwap(int &a, int &b, int &c)` that performs a right cyclic shift such that `a` receives the value of `c`, `b` receives `a`, and `c` receives `b` without returning any value. Test the function in `main()` using user inputs inside a loop until the user inputs 0.

Pass-by-Reference | While Loop | Modular Functions

Sample Input	Sample Output
`a = 1, b = 2, c = 3`	`After Shift: a = 3, b = 1, c = 2`

Level 2: Intermediate Applications (Functions, Loops & Arrays)

Problem 4: Dual-Result Array Analyzer

Arrays | Pass-by-Reference | Loops

Create a program that reads an integer array of size N. Implement a function `void getArrayStats(const int arr[], int size, int &minVal, int &maxVal, double &avgVal)`. The function must compute the minimum element, maximum element, and average value in a single pass, updating the caller's variables via reference parameters.

```
void getArrayStats(const int arr[], int size, int &minVal, int &maxVal, double &avgVal); // Const reference / const array ensures source data immutability while returning 3 results!
```

Sample Input	Sample Output
`Array: [14, 5, 89, -3, 22]`	`Min: -3 | Max: 89 | Avg: 25.40`

Problem 5: In-Place Array Filter & Transformer

Array Modification | Range-Based Loop | References

Write a program that takes an array of N integers. Use a range-based `for` loop with reference elements (`for (int &num : arr)`) to modify the array in-place: if an element is even, double it; if odd, square it. Print the updated array from `main()` to demonstrate that modifications directly updated original storage.

Range-based For Loop | Reference Loop Variable | In-Place Mutation

Sample Input	Sample Output
`Input Array: [2, 3, 4, 5]`	`Output Array: [4, 9, 8, 25]`

Problem 6: String Vowel Sanitizer & Character Counter

Strings | Pass-by-Reference | Character Manipulation

Implement a function `int sanitizeString(std::string &str, char replacementSymbol)`. The function should iterate over the reference string `str`, replace every vowel (case-insensitive) with `replacementSymbol`, and return the total count of replaced characters. Display the modified string in `main()`.

std::string References | Case Conversion | Side Effects in Functions

Sample Input	Sample Output
`str = "Hello World", symbol = '*'`	`Returned Count: 3` `Modified String: "H*ll* W*rld"`

Level 3: Advanced Concepts (Return-by-Reference & Const References)

Problem 7: Subscript L-Value Alias Generator

Return-by-Reference | Array Indexing | L-Value

Write a function `int& getElementAt(int arr[], int index, int size)` that returns a reference to the array element at the given index. If the index is out of bounds, print an error and return a reference to a static fallback error variable. In `main()`, use this function on the left side of an assignment operator (as an L-value) to modify array values.

```
      // Demonstration of Return-by-Reference as L-Value: getElementAt(myArray, 2, 5) = 99; // Directly updates myArray[2] to 99!
```

Sample Operation	Resulting State
`Array: [10, 20, 30] -> call getElementAt(arr, 1, 3) = 500`	`Array updated to: [10, 500, 30]`

Problem 8: Const Reference Performance vs Safety Matrix

Const References | Temporary Objects | Efficiency

Construct a program to benchmark passing large `std::string` objects. Write two functions: `void processByValue(std::string s)` and `void processByConstRef(const std::string &s)`. Demonstrate that `const std::string &` prevents accidental modification while avoiding copy overhead. Additionally, show how `const double &ref = 10.5;` extends the lifetime of a temporary literal value.

Const References | R-Value Binding to Const Ref | Performance Optimization

Code Snippet	Behavior Analysis
`const int &r = 50;`	`Valid: Const reference extends lifetime of temporary integer literal 50.`

Problem 9: Cumulative Challenge: Bank Account Batch Ledger

Comprehensive Integration | References | Structs/Arrays

Design a mini banking ledger system. Define a structure `Account` with `id` and `balance`. Implement functions: 1. `bool findAccount(Account accs[], int size, int targetID, Account* &foundAcc)` 2. `void applyMonthlyInterest(Account &acc, double rate, double &totalInterestPaid)` Process a list of 5 accounts in `main()`, apply interest using pass-by-reference, update total interest tracked across all accounts, and print final balances.

Struct References | Reference to Pointer | Cumulative Module Capstone

Initial Balance	Interest Rate	Final Balance & Interest Tracker
`Acc #101: $1,000.00`	`5.0%`	`Acc Balance: $1,050.00 | Total Interest Pool Updated`